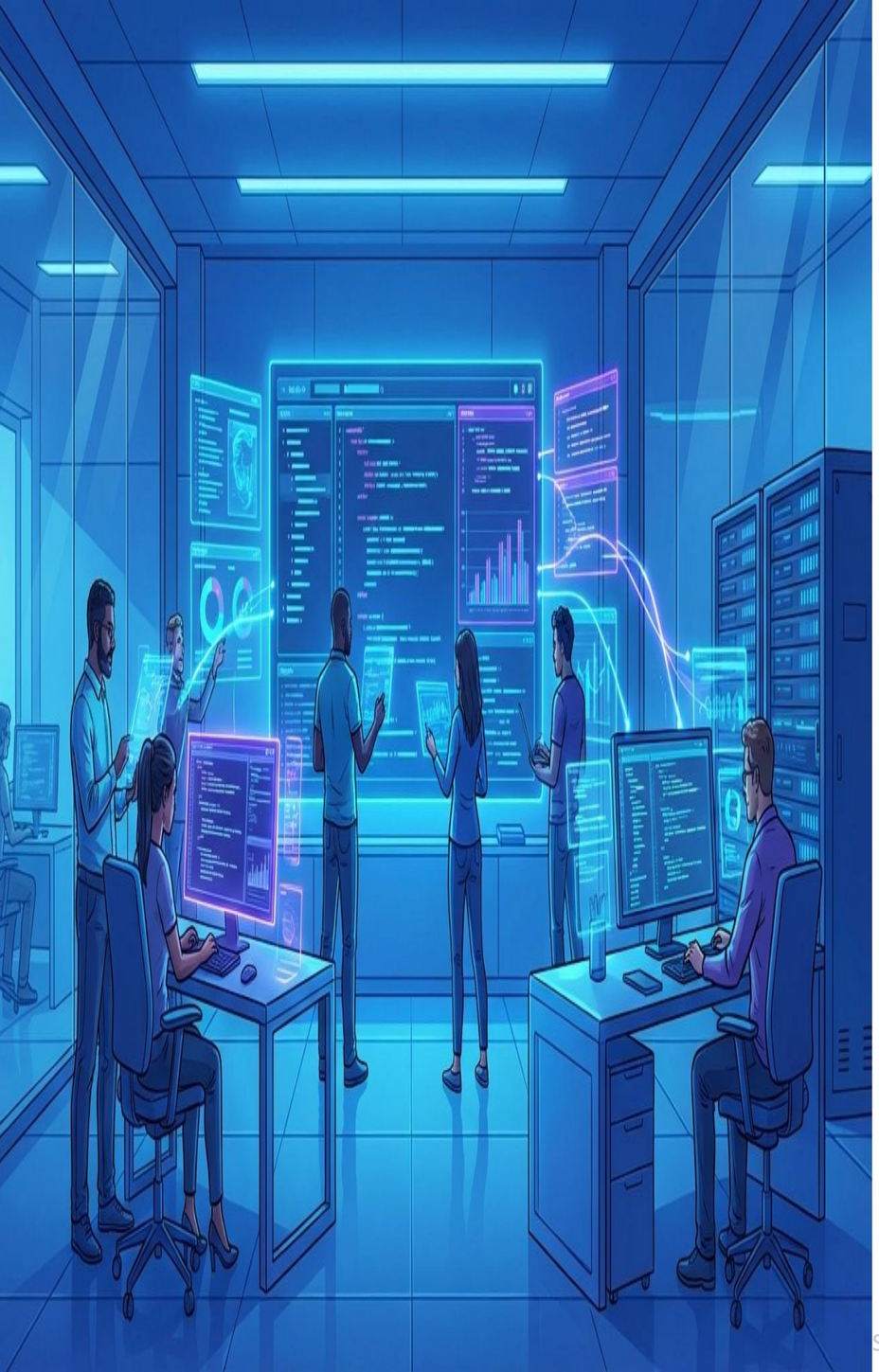


Programmation : Structuration de Projet et Collaboration

Techniques de Développement Logiciel

Par **Hermann Agossou**, Doctorant - UTER MID@S - ECC / UH2C

École Centrale Casablanca

Coding Week

1st Year - Eng School - 2026

Pourquoi la Programmation Collaborative Compte ?

Problèmes Courants en Équipe

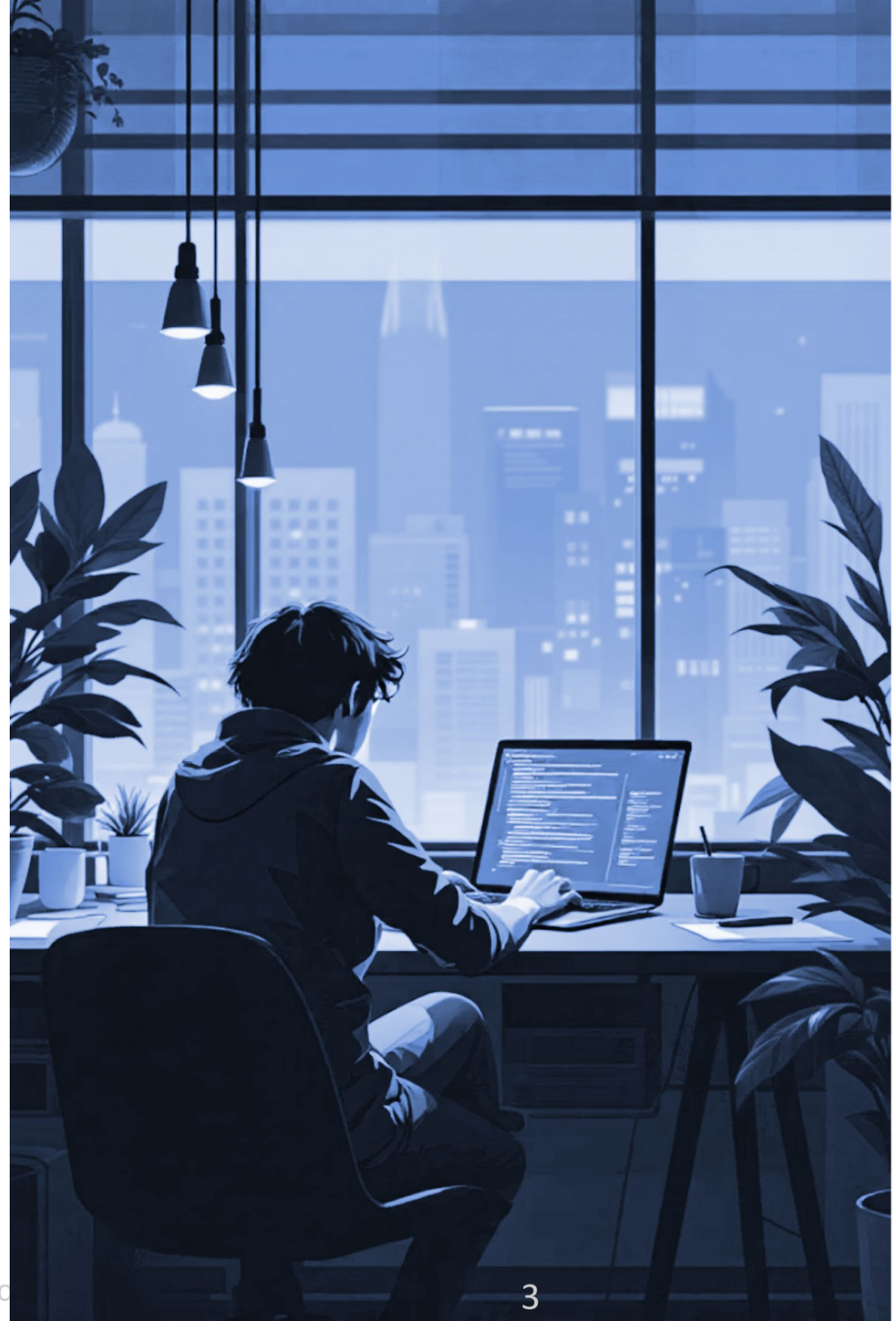
- Code écrasé par des modifications concurrentes
- Structure de projet chaotique
- Formats et styles incohérents
- Dépendances non maîtrisées
- Difficultés majeures lors du débogage
- …

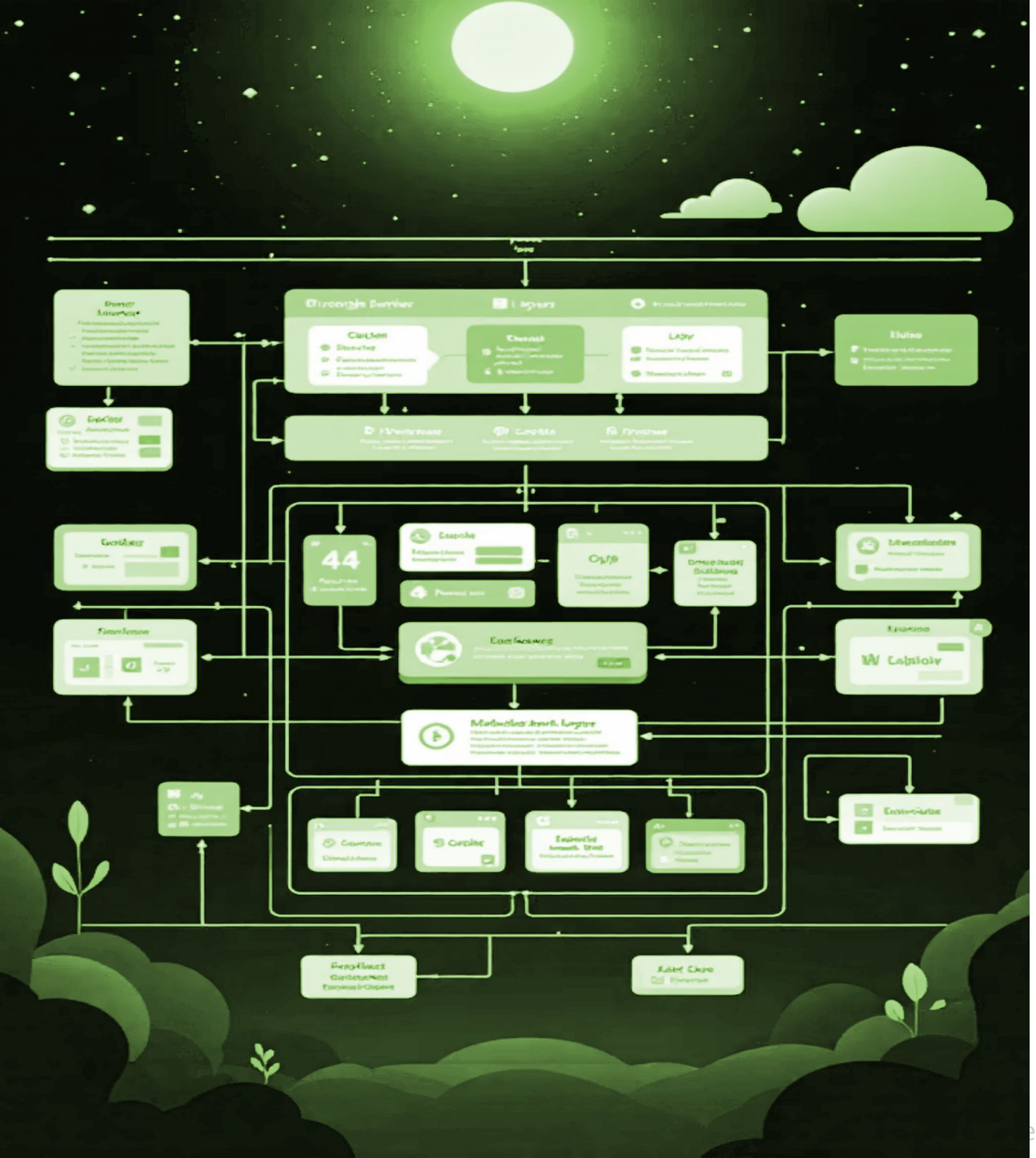
Solutions Professionnelles

- Git et contrôle de version
- Architecture modulaire
- Tests automatisés
- Intégration et déploiement continus (CI/CD)
- …

Objectifs

- CONNAÎTRE LES FONDAMENTAUX DE CONCEPTION DES LOGICIELS
- APPRENDRE LES MEILLEURES PRATIQUES DE QUALITÉ DE CODE
- INTRODUCTION AU CONTRÔLE DE VERSION





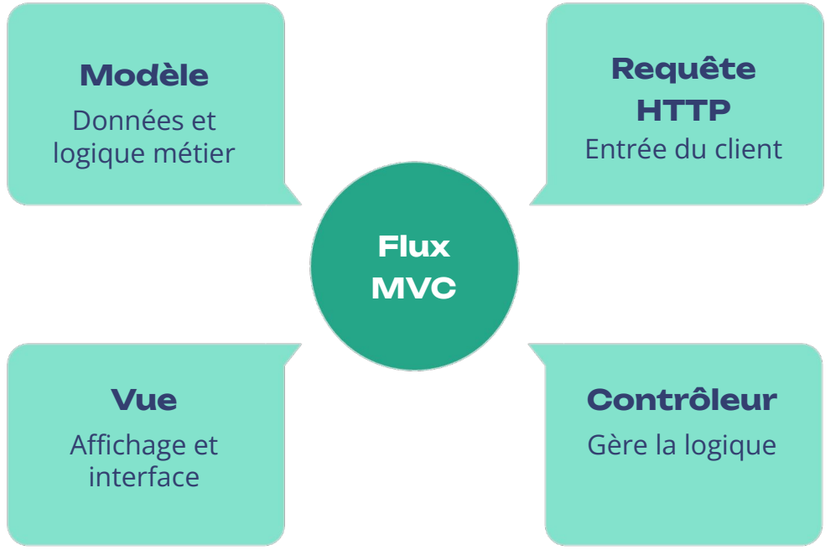
I - Conception Logicielle et Structure

Modèle – Vue – Contrôleur

Le MVC (Modèle-Vue-Contrôleur) est un design pattern utilisé dans la conception logicielle pour structurer les interfaces utilisateur, les données et la logique de contrôle.

Le design pattern MVC comporte trois parties :

1	2	3
Modèle Gère les données et la logique métier.	**Vue** Gère la mise en page et l'affichage.	**Contrôleur** Achemine les commandes vers le modèle et les éléments de la vue.



Le diagramme illustre comment une requête HTTP atteint d'abord le Contrôleur, qui orchestre ensuite les interactions avec le Modèle (pour les données et la logique) et la Vue (pour la présentation), facilitant ainsi l'échange de données pour fournir le résultat final.

Modèle – Vue – Contrôleur

Voici un exemple d'architecture de fichiers MVC pour une application web typique en Python :

```
car_management/
├── app.py                    # Point d'entrée principal (lance l'application)
├── models/
│   ├── __init__.py          # Rend le répertoire models comme un paquet
│   ├── car.py               # Classe modèle de voiture
│   └── car_model.py         # Gère le stockage et les opérations sur les données
├── views/
│   ├── __init__.py          # Rend le répertoire views comme un paquet
│   └── car_view.py          # Gère l'affichage des données
├── controllers/
│   ├── __init__.py          # Rend le répertoire controllers comme un paquet
│   └── car_controller.py    # Gère la logique métier et les actions utilisateur
├── tests/
│   ├── test_car.py          # Tests unitaires pour le modèle de voiture
│   ├── test_car_model.py    # Tests unitaires pour la gestion des données
│   └── test_car_controller.py # Tests unitaires pour la logique du contrôleur
├── requirements.txt/pyproject.toml # Dépendances
├── Dockerfile                    # Configuration du conteneur Docker
├── docker-compose.yml            # Configuration Docker compose
└── README.md                     # Documentation
```

Architecture logicielle

Exemple pour un projet de science des données

```
car_price_prediction/
├── data/                      # Jeux de données bruts et/ou nettoyés
├── notebooks/                 # Notebooks Jupyter (EDA, entraînement)
├── src/                       # Répertoire du code source
│   ├── dataloader.py         # Utilitaires de chargement
│   ├── models.py             # Définitions des modèles
│   ├── config.py             # Paramètres de configuration
│   ├── utils/                # Fonctions auxiliaires
│   │   ├── data_preprocessing.py
│   │   ├── feature_engineering.py
│   │   └── evaluation.py
│   ├── app.py                # Inférence (Streamlit/Flask)
│   ├── main.py               # Script d'entraînement/tests
│   ├── models/               # Modèles sauvegardés
│   │   ├── trained_model.pkl
│   │   └── checkpoints/
├── tests/                    # Tests unitaires
│   ├── test_dataloader.py
│   ├── test_model.py
│   └── test_utils.py
├── reports/                  # Rapports et visualisations
│   ├── figures/
│   └── summary.md
├── requirements.txt/pyproject.toml # Dépendances
├── Dockerfile                      # Configuration Docker
├── docker-compose.yml              # Configuration multi-conteneurs
└── README.md                       # Documentation
```

Verrouillage des dépendances

Lors de la maintenance d'un projet, vous ne pouvez pas toujours ignorer la maintenance de ses dépendances !

Pourquoi est-ce important ?

- Définir les dépendances et les contraintes de version par projet 🛠️
- Prévient les conflits de version entre les projets 🔄
- Assure la reproductibilité 🔒

Bonnes pratiques !

✅ Verrouillez toujours les dépendances avant de partager

✅ Utilisez Docker pour un contrôle complet de l'environnement

✅ Gardez les dépendances minimales et à jour

✅ Utilisez .gitignore pour exclure venv/ du contrôle de version

✅ Utilisez un outil moderne tel que uv (de Astral) pour vous simplifier ces tâches

II - Qualité du Code et Gestion des Erreurs

Tests, Refactoring et Prévention des Bugs

Lisibilité et maintenabilité du code

Nommage

Noms explicites pour les variables, fonctions, méthodes et classes

Docstrings

Documentez les méthodes à l'aide de commentaires et de docstrings

Responsabilité unique

Gardez les fonctions courtes et ciblées

Modules

Divisez les fichiers volumineux en modules

🚫

```python
def calc(x, y):  # What does this calculate?
    return x * y
```

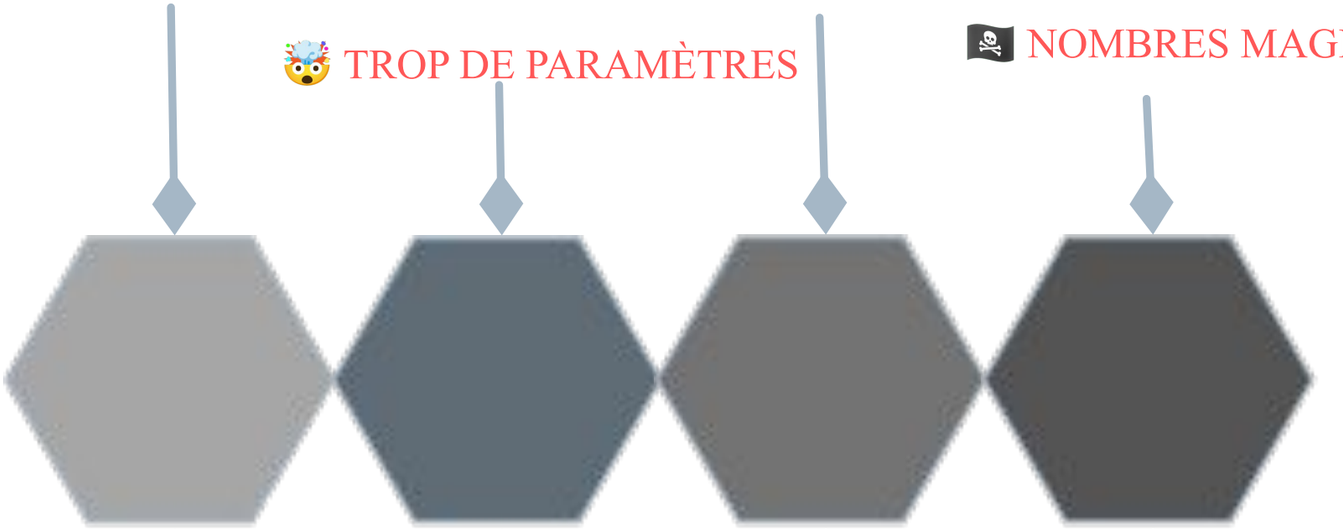
✅

```python
def calculate_area(length: float, width: float) -> float:
    """Calculates the area of a rectangle given its length and width.
    """
    return length * width
```

Code Smells et Refactoring

🚫 **AVOID:**

- 🧨 FONCTION TROP LONGUE
- 🤯 TROP DE PARAMÈTRES
- 🔁 LOGIQUE DUPLIQUÉE
- ☠️ NOMBRES MAGIQUES



✅ **BETTER**

- DÉCOUPER EN PLUS FONCTIONS COURTES ET CIBLÉES
- UTILISER DES OBJETS/DATACLASSES POUR STRUCTURER LES DONNÉES
- EXTRAIRE DES FONCTIONS/MÉTHODES RÉUTILISABLES
- REMPLACER PAR DES CONSTANTES NOMMÉES

Erreurs de Syntaxe

QU'EST-CE QUI NE VA PAS ?

- Caractères mal placés (parenthèses, guillemets non fermés)
- Indentation incorrecte
- Mots-clés mal orthographiés

CONSÉQUENCE

- Le code ne peut pas être exécuté du tout.
- L'interpréteur ou le compilateur refuse de lancer le programme

COMMENT ÉVITER

- Utiliser un IDE avec vérification syntaxique en temps réel
- Linters : pylint, flake8 (Python), ruff, ESLint (JavaScript)
- Outils de formatage : black, prettier, ruff

Exemple : Parenthèse manquante

✗ **Avant (Erreur de syntaxe)**

```
print("Bonjour"
# SyntaxError: unexpected EOF while parsing
```

✓ **Après (Corrigé Manuellement)**

```
print("Bonjour")
# Output: Bonjour
```

Erreurs sémantiques statiques

QUE SE PASSE-T-IL ?

- Utilisation de types de données inappropriés (int + str)
- Fonctions retournant des types inattendus

CONSÉQUENCE

- Plantages à l'exécution ou comportement inattendu

COMMENT ÉVITER

- Utiliser des annotations de type et des outils d'analyse statique
- Outils Python : mypy, pyright

Exemple : Incohérence de type

Avant (sans annotations de type, erreur potentielle) :

```
def add_numbers(a, b):
    return a + b

result = add_numbers(10, "hello") # Cela provoquerait un
TypeError à l'exécution
print(result)
```

Après (avec annotations de type, l'analyse statique peut détecter l'erreur) :

```
def add_numbers_typed(a: int, b: int) -> int:
    return a + b

# mypy ou pyright signaleraient la ligne suivante comme une
erreur avant l'exécution
# result_typed = add_numbers_typed(10, "hello")

# Utilisation correcte :
result_typed = add_numbers_typed(5, 7)
print(result_typed)
```

Erreurs de logique

QU'EST-CE QUI NE VA PAS ?

- Formules incorrectes, mauvaises conditions, boucles erronées

CONSÉQUENCE

- Le code s'exécute mais produit des résultats incorrects

COMMENT ÉVITER

- Écrire des tests unitaires pour vérifier la logique
- Tests continus (CI/CD)
- Outils Python : pytest, unittest

Exemple : Calcul incorrect (Logique erronée)

```
def calculer_surface_fausse(longueur, largeur):
    # Erreur logique : devrait être longueur * largeur
    return longueur + largeur

l = 5
la = 10
print(f"Surface erronée : {calculer_surface_fausse(l, la)}")
# Attendu : 50, Réel : 15
```

Exemple : Calcul corrigé (Logique corrigée)

```
def calculer_surface_correcte(longueur, largeur):
    return longueur * largeur

l = 5
la = 10
print(f"Surface correcte : {calculer_surface_correcte(l, la)}")
# Attendu : 50, Réel : 50
```

Erreurs en Runtime & Gestion d'Exceptions

QU'EST-CE QUI NE VA PAS ?

- Accès à un index inexistant
- Division par zéro
- Appel de méthode sur un objet None
- Fichier non trouvé lors de l'ouverture

CONSÉQUENCE

- Le programme s'arrête de façon inattendu.
- Les exceptions non gérées conduisent à une mauvaise expérience utilisateur et peuvent corrompre des données en cours de traitement.

COMMENT ÉVITER

- *Anticiper, vérifier, retourner tôt — le pattern Return Early:* Le Return Early Pattern consiste à vérifier les conditions problématiques au début de la fonction et retourner immédiatement si elles sont invalides. Cela réduit considérablement la complexité du code et évite les imbrications profondes de blocs conditionnels.

Exemple : Return Early

```
def traiter_utilisateur(user_id: Optional[int]):
    if user_id is None or user_id <= 0:
        return None
        # Sortie précoce

    user = get_user(user_id)
    if not user:
        return None
        # Sortie précoce

    return user.process()
```

💡 Chaque vérification élimine un cas invalide, simplifiant le flux principal.

Erreurs en Runtime & Gestion d'Exceptions

GESTION DES EXCEPTIONS

Anticiper les erreurs possibles et capturer les exceptions gracieusement. Cela permet au programme de continuer son exécution ou d'échouer proprement, en informant l'utilisateur de manière claire et en évitant toute corruption de données.

LOGGING — Traçabilité

Enregistrer les événements importants avec différents niveaux de sévérité (DEBUG, INFO, WARNING, ERROR, CRITICAL). Un bon système de logs facilite le débogage en production et fournit un historique précieux pour l'analyse post-mortem des incidents.

MONITORING — Surveillance

Surveiller l'application en temps réel grâce à des alertes automatiques sur anomalies. La collecte de métriques (performance, taux d'erreurs, latence) permet une détection proactive des problèmes avant qu'ils n'affectent les utilisateurs.

Exemple complet en Python

```python
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def diviser(a, b):
    if b == 0:
        logger.error(f'Division par zéro')
        raise ValueError('Diviseur = zéro')
    result = a / b
    logger.info(f'{a}/{b} = {result}')
    return result
```

Try / Except

Le bloc try/except capture les erreurs et permet au programme de réagir intelligemment au lieu de crasher

Monitoring & Alertes

Connectez votre logger à un système de monitoring pour recevoir des alertes automatiques et surveiller les métriques en temps réel.

Tests

Pourquoi les tests sont-ils importants ?

- **Fiabilité**: Garantit la fiabilité et la maintenabilité du code
- **Prévient les régressions**: Empêche les régressions lors de la mise à jour du code
- **Accélère le développement**: Améliore le débogage et accélère le développement

Principaux types de tests :

01

Tests unitaires
Testent des fonctions ou méthodes individuelles

02

Tests d'intégration
Testent plusieurs composants ensemble

03

Tests de bout en bout (E2E)
Simulent des scénarios réels d'utilisation

Écrire des tests unitaires avec pytest

Les tests unitaires sont essentiels pour garantir la justesse et la fiabilité des composants individuels de votre base de code. Nous utiliserons `pytest`, un framework de test Python populaire, pour démontrer comment écrire et exécuter des tests.

Exemple de fonction : `add()`

Considérons une fonction simple `add` avec des indices de type et une docstring :

```
def add(a: int, b: int) -> int:
    """Additionne deux entiers et retourne la somme."""
    return a + b
```

Écrire des tests unitaires avec pytest

Les tests unitaires sont essentiels pour garantir la justesse et la fiabilité des composants individuels de votre base de code. Nous utiliserons `pytest`, un framework de test Python populaire, pour démontrer comment écrire et exécuter des tests.

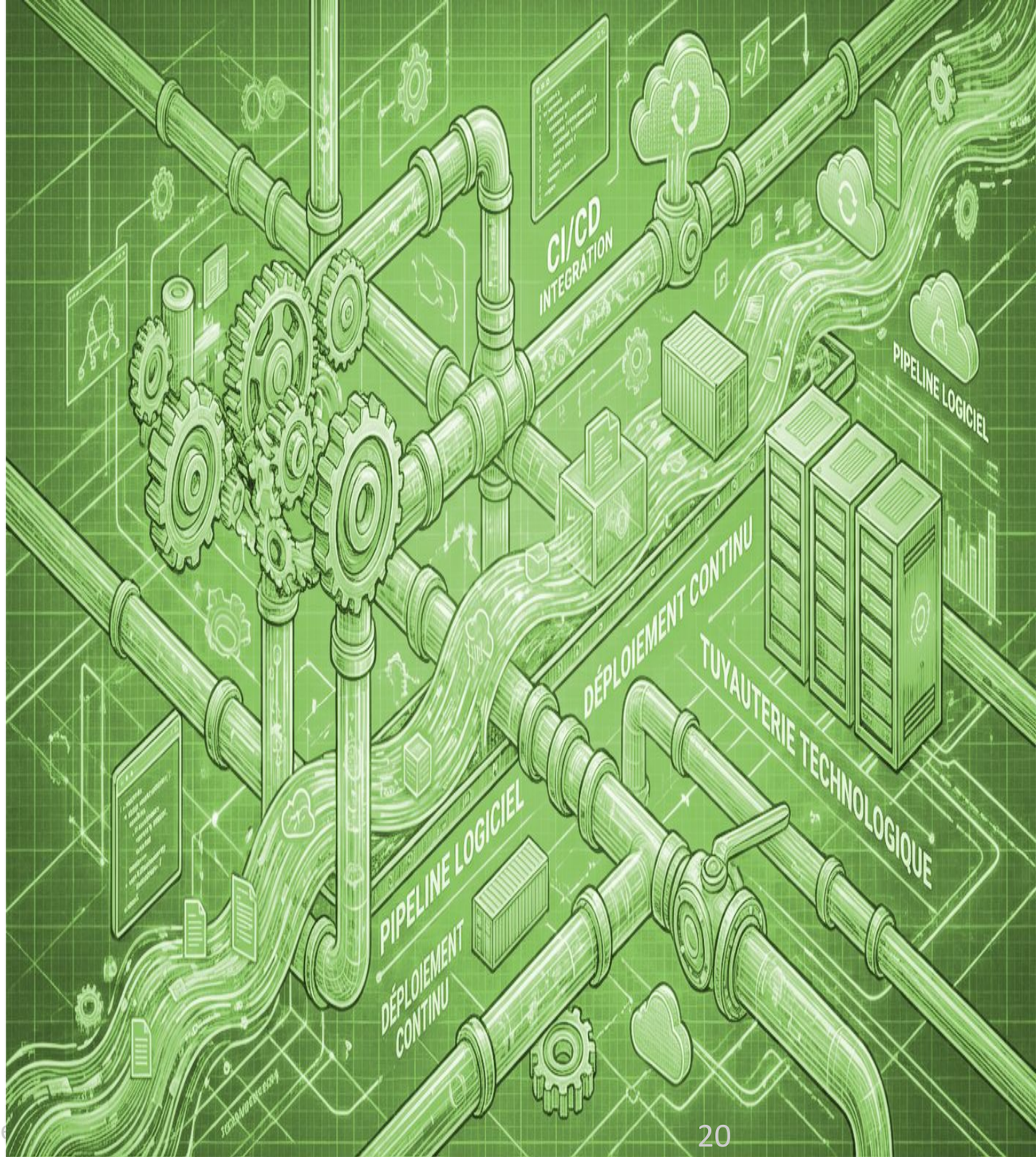
Fichier de test (`tests/test_math_operations.py`)

Voici comment vous pourriez écrire des tests unitaires pour la fonction `add` en utilisant `pytest`. Nous inclurons des tests pour des entrées valides et une vérification simple pour une entrée invalide.

```
import pytest
from your_module import add

def test_add_nombres_positifs():
    assert add(1, 2) == 3

def test_add_nombres_negatifs():
    assert add(-1, -2) == -3

def test_add_zero():
    assert add(0, 5) == 5
    assert add(-5, 0) == -5

def test_add_type_entree_invalide():
    with pytest.raises(TypeError):
        add("1", 2)
```

III - Collaboration et contrôle de version

Git, GitHub et flux de travail collaboratifs



Contrôle de version : 3 approches

Le contrôle de version est un système qui enregistre les modifications apportées à un fichier ou à un ensemble de fichiers au fil du temps, afin que vous puissiez rappeler des versions spécifiques ultérieurement. C'est un outil indispensable pour les développeurs individuels et, surtout, pour les équipes collaboratives.

Les trois types :

- **Local** : archivé localement (ancien, risqué)
- **Centralisé** : serveur central SVN (point de défaillance unique)
- **Distribué** : Git (flexible, robuste, standard de l'industrie)

Pourquoi est-ce essentiel pour les projets d'équipe :

- Suivi des changements (modifications, auteurs, dates).
- Travail en équipe à distance.
- Gestion des conflicts

Git – La référence de l'industrie

Git est un système de contrôle de version distribué devenu le standard de l'industrie.

Ses avantages clés :

```
root@b6ec894e0011:/myapp# touch .gitignore
root@b6ec894e0011:/myapp# git add .
root@b6ec894e0011:/myapp# git commit -m "add gitignore"
[master 4b3f79b] add gitignore
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .gitignore
root@b6ec894e0011:/myapp#
```

Vitesse et flexibilité

Git excelle en vitesse et flexibilité de branching/fusion.

Pilier de l'industrie

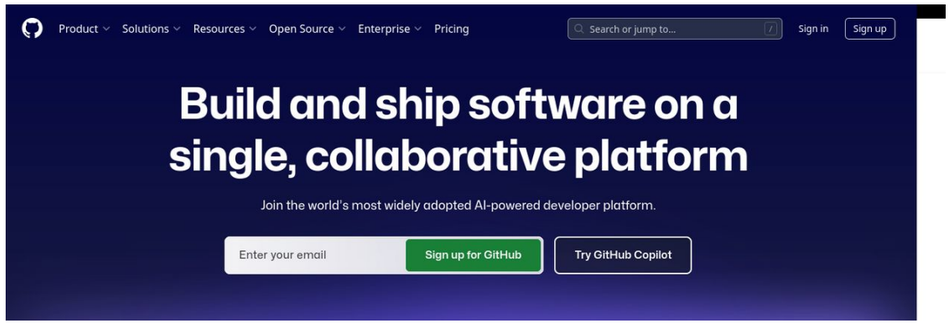
Technologie sous-jacente de GitHub, GitLab et Bitbucket.

Nature distribuée

Chaque contributeur dispose d'une copie locale complète, permettant le travail hors ligne.

GitHub – Hébergeur Web pour dépôts Git

Plateforme cloud pour héberger et gérer des dépôts Git avec Pull Requests, revues de code et suivi des tickets.



Pourquoi utiliser GitHub :

Hébergement de contrôle de version
Hébergement centralisé et sécurisé pour les dépôts Git.

Fonctionnalités de collaboration
Pull Requests, revues de code et suivi des tickets pour collaborer efficacement.

Intégration CI/CD
Automatisation via GitHub Actions pour construction, test et déploiement.

Foyer des projets open source avec une communauté dynamique de développeurs.

Comment Git et GitHub fonctionnent ?

Comprendre les composants principaux et le flux de travail de Git et GitHub est essentiel pour un contrôle de version efficace et un développement collaboratif.

Composants principaux :

Espace de travail (Workspace)

Votre ordinateur local où vous modifiez activement les fichiers et écrivez le code.

Index / Zone de préparation (Staging Area)

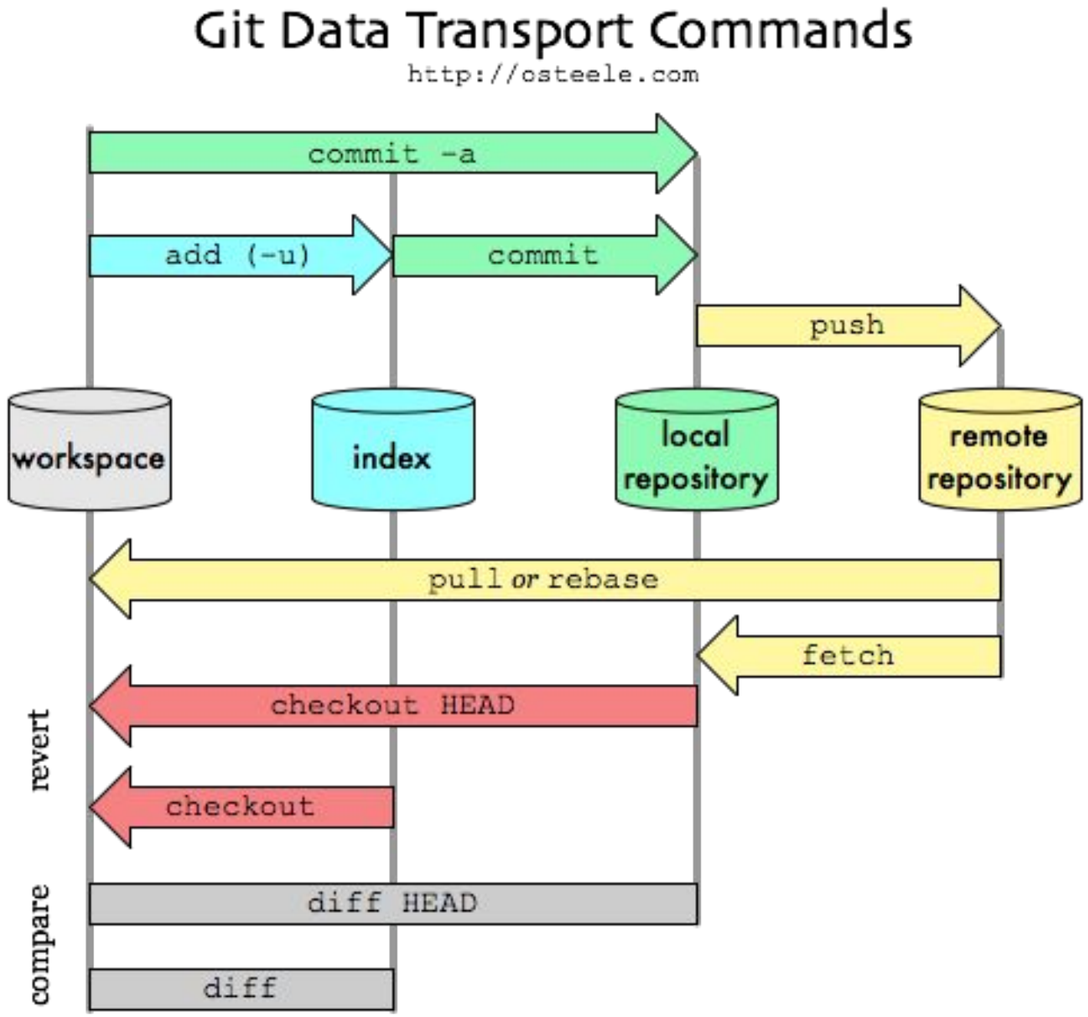
Une zone de « brouillon » où vous préparez les changements avant de les valider (commit). Elle contient une image instantanée des modifications que vous souhaitez inclure dans votre prochaine validation.

Dépôt local (Local Repository)

L'historique complet de toutes vos validations, stocké sur votre machine locale. Il suit toutes les modifications apportées à votre projet.

Dépôt distant (Remote Repository)

Une copie du dépôt de votre projet hébergée sur un serveur (comme GitHub). Il sert de centre névralgique pour la collaboration en équipe et la sauvegarde.



Comment Git et GitHub fonctionnent ?

git init

Initialiser un nouveau dépôt Git dans le répertoire courant. Cette commande crée un sous-dossier .git qui contient tous les fichiers nécessaires au fonctionnement du dépôt.

git status

La commande git status affiche l'état de l'arbre de travail et de la zone de staging. Elle vous permet de voir quels changements ont été préparés, lesquels ne l'ont pas été, et quels fichiers ne sont pas suivis par Git. C'est une commande de diagnostic essentielle que vous utiliserez fréquemment tout au long de votre workflow.

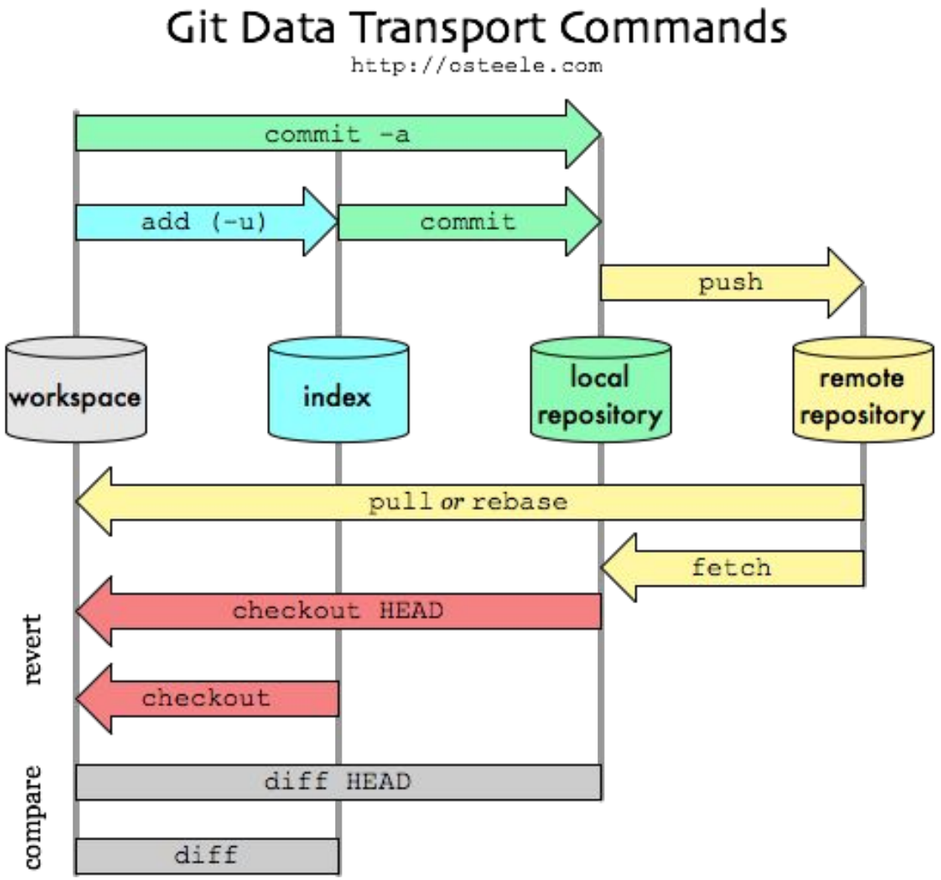
git clone <URL>

Cloner un dépôt existant depuis une URL distante. Cette commande crée une copie complète du dépôt, y compris tout l'historique des commits, les branches et les tags. Idéal pour rejoindre un projet existant.

git add .

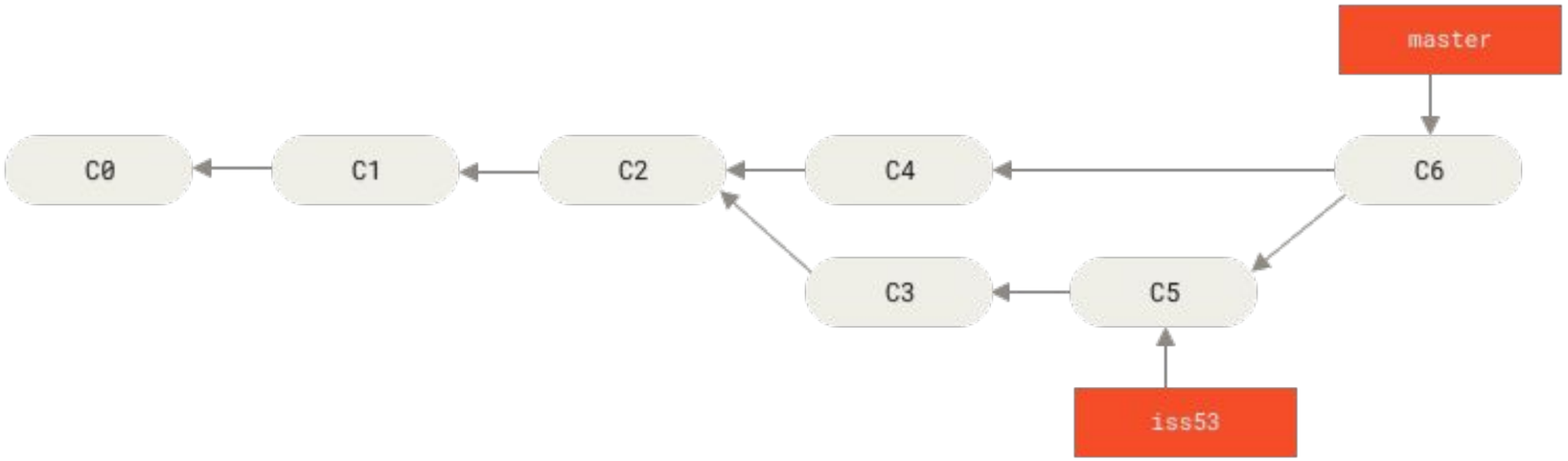
La commande git add prépare vos fichiers modifiés pour le prochain commit. En utilisant "git add .", vous ajoutez toutes les modifications du répertoire courant. Vous pouvez aussi cibler des fichiers spécifiques avec "git add <fichier>". Cette étape intermédiaire vous donne un contrôle précis sur ce qui sera inclus dans votre prochain commit.

Workflow Git simple pour travailler sur un projet depuis son ordinateur

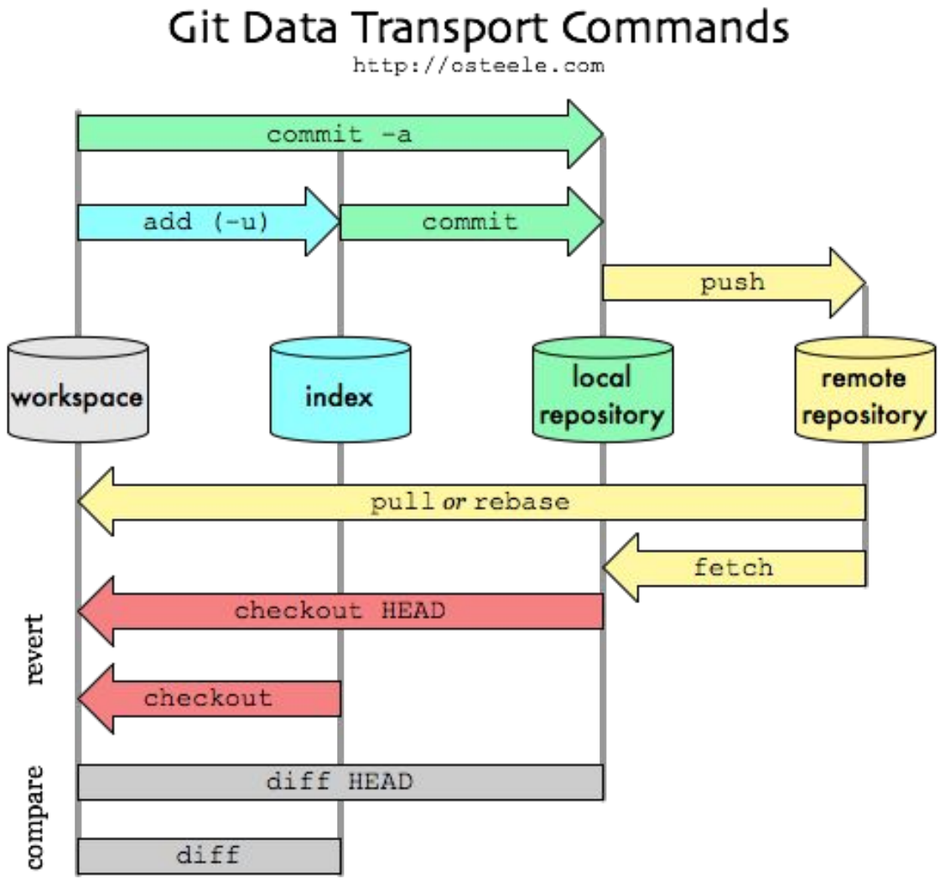



```
# Cloner le dépôt depuis GitHub (copie du projet sur ta machine)
git clone https://github.com/hermann-web/project.git

# Entrer dans le dossier du projet
cd project

# Ou démarre un nouveau dépôt local
cd project
git init

# Modifier le code dans ton IDE
# ... les modifications des fichiers se font ici ...

# Voir quels fichiers ont été modifiés
git status

# Ajouter les fichiers modifiés dans la zone de préparation (staging)
git add .

# Enregistrer un snapshot des modifications avec un message clair
git commit -m "Ajout du module d'authentification utilisateur"

# Mettre à jour le projet local avec la dernière version du dépôt distant
git push
# Pour un nouveau dépôt local
gir push -u main
```

Git Workflow en Équipe

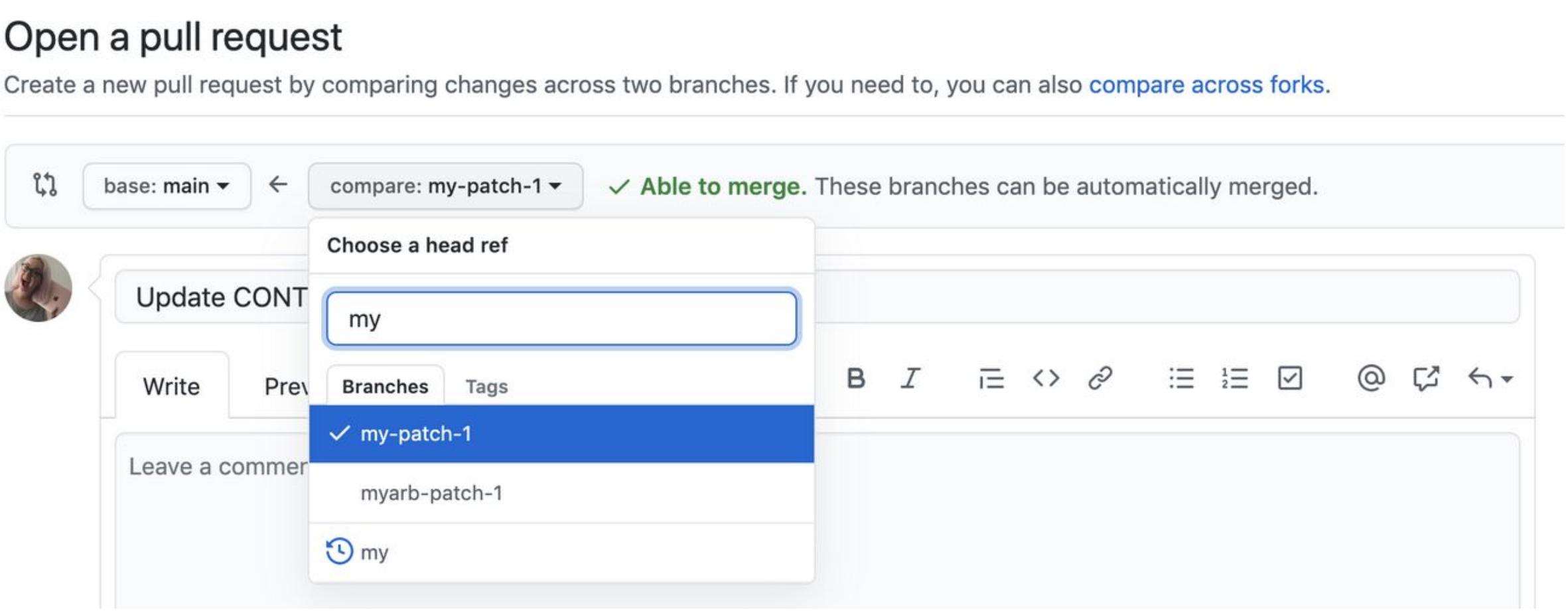


Workflow Git simple pour travailler sur un projet d'équipe depuis son ordinateur




```
# Cloner le dépôt depuis GitHub (copie du projet sur ta machine)
git clone https://github.com/org/project.git

# Entrer dans le dossier du projet
cd project

# Créer une nouvelle branche pour développer une fonctionnalité
git checkout -b feature-login

# Modifier le code dans ton IDE
# ... les modifications des fichiers se font ici ...

# Voir quels fichiers ont été modifiés
git status

# Ajouter les fichiers modifiés dans la zone de préparation (staging)
git add .

# Enregistrer un snapshot des modifications avec un message clair
git commit -m "Ajout du module d'authentification utilisateur"

# Envoyer la branche sur GitHub pour qu'elle soit revue par l'équipe
git push origin feature-login

# Après la revue de code et la fusion (merge) dans la branche principale :
git checkout main

# Mettre à jour le projet local avec la dernière version du dépôt distant
git pull origin main
```

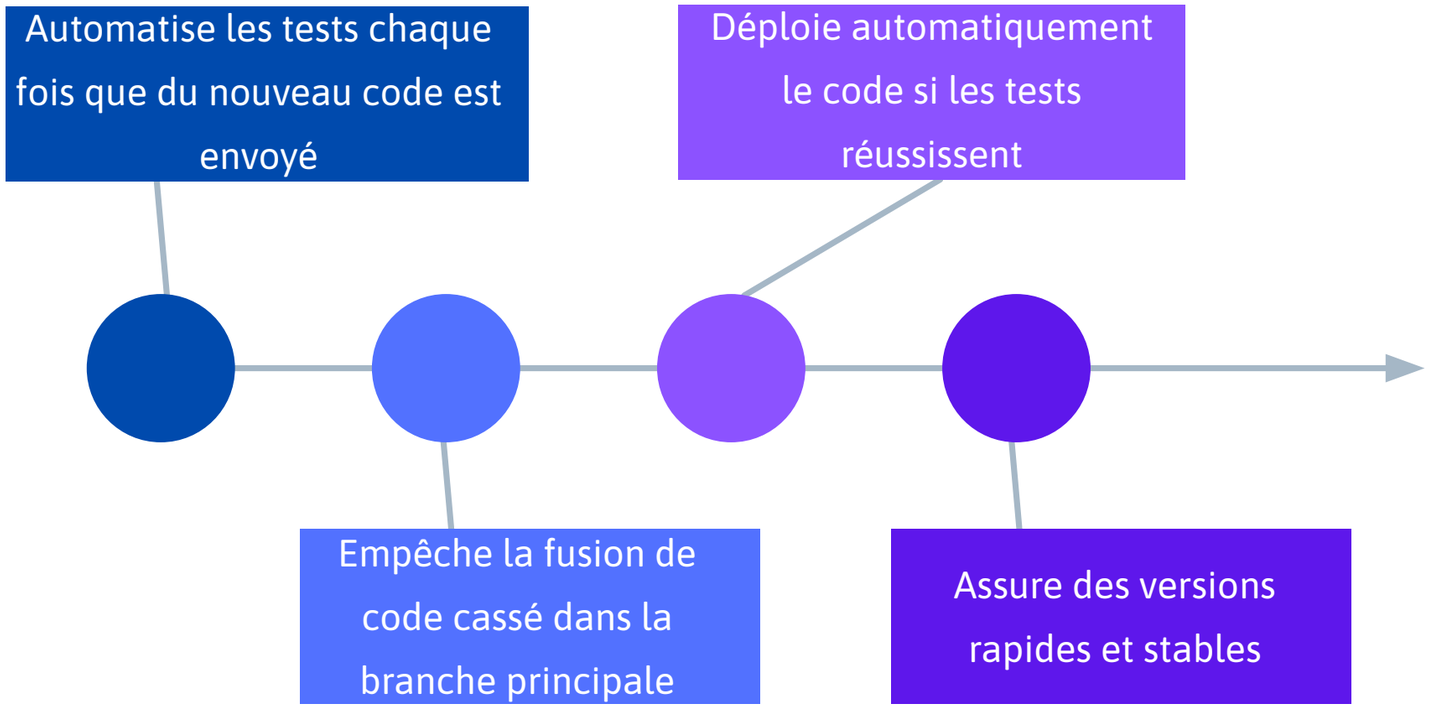
Le Pull Request



Le CI/CD

Intégration Continue (CI)

Déploiement Continu (CD)




```yaml
name: Run Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v3
        with:
          python-version: "3.10"

      - name: Install dependencies
        run: pip install pytest

      - name: Run tests
        run: pytest tests/
```

🛠️ **Tools** GitHub Actions, GitLab CI, Jenkins, ...

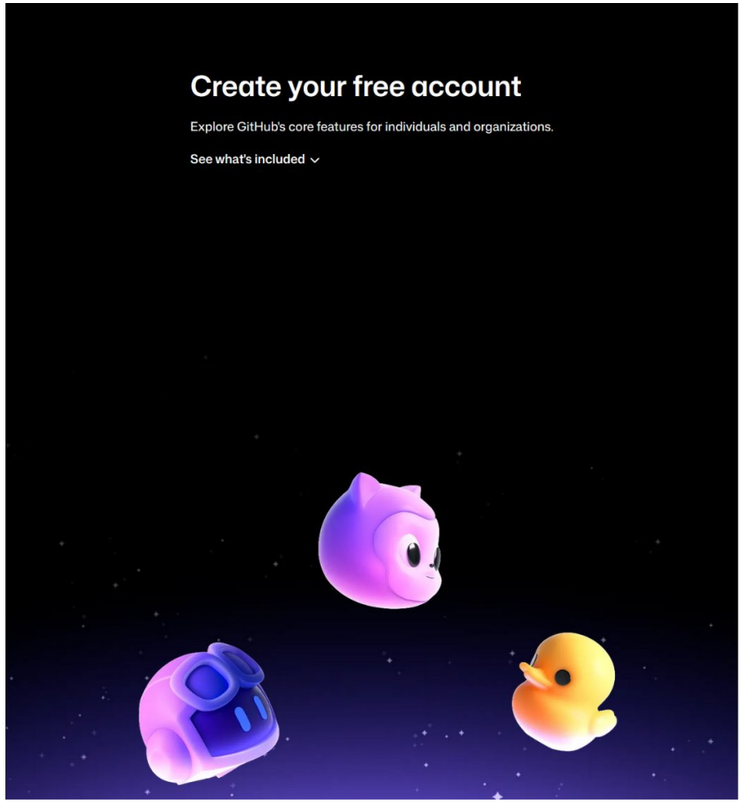
Un exemple de configuration CI avec GitHub Actions

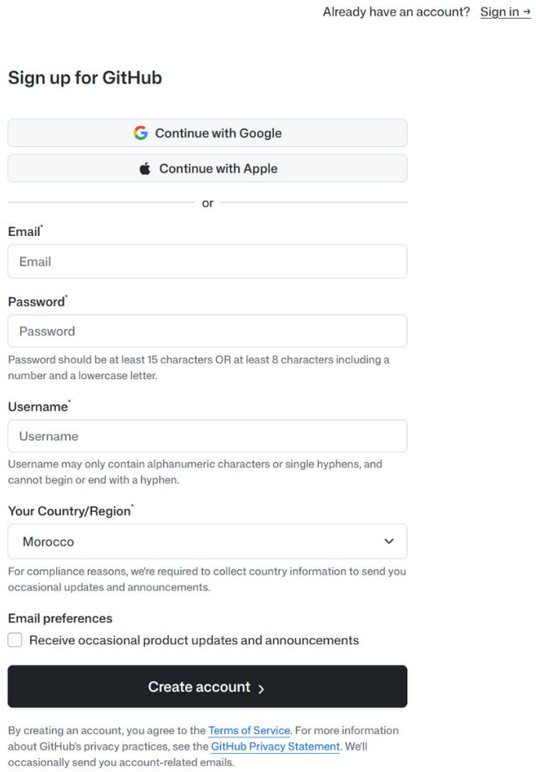
- Désormais, GitHub exécutes les tests automatiquement à chaque commit 🚀

Annexe

Installer Git et Créer un compte sur Github

Créez un compte github sur https://github.com/signup si ce n'est pas encore fait





Verrouillage des dépendances en python

Un ensemble d'outils conçus pour la gestion des dépendances

Outil	Cas d'usage	Lancer un projet	Verrouillage	Type de verrou
venv	Intégré, léger	python -m venv venv	requirements.txt	Basé sur un snapshot
virtualenv	Plus rapide, meilleur support des anciennes versions Python	virtualenv venv	requirements.txt	Basé sur un snapshot
pipreqs	Générer requirements.txt à partir des imports	pipreqs .	requirements.txt	Basé sur des contraintes
pipenv	Gère venv + dépendances	pipenv install	Pipfile / Pipfile.lock	Verrou complet des dépendances
conda	Gestion de Python et des dépendances non-Python	conda create -n myenv	environment.yml	Basé sur des contraintes
poetry	Moderne, gère venv + packaging	poetry init / poetry install	pyproject.toml / poetry.lock	Verrou complet des dépendances
uv	**Moderne, alternative très rapide à pip + venv ou poetry !**	**uv init / uv sync**	**requirements.txt ou pyproject.toml / uv.lock**	**Verrou complet des dépendances (uv>=0.3.0)**

Installation d'outils pratiques

Logiciel	Site web	Windows (winget)	Microsoft Store	Linux (gestionnaire de paquets)
GitHub Desktop	https://desktop.github.com/download/	winget install GitHub.GitHubDesktop	N/A	Linux is not supported yet as per official documentation Official documentation. You can install a fork from https://github.com/shiftkey/desktop
Git	https://git-scm.com/install/	winget install Git.Git	N/A	Command available on the website
Visual Studio Code	https://code.visualstudio.com/	winget install Microsoft.VisualStudioCode	VSCode	sudo snap install code --classic / sudo apt install code (selon distro)
Python	https://www.python.org/downloads/	winget install python	Python3	Prefer using python through uv
uv (gestionnaire de packages Python)	https://docs.astral.sh/uv/	winget install astral-sh.uv	N/A	Command available on the website